

Unit -7 : Advanced Android Concepts [10 HRS]

Local database with SQLite (Establishing connection, creating database and tables, data manipulation)

Introduction to Local Databases in Android

Local databases allow apps to store data directly on the user's device, providing offline access and faster data retrieval compared to network-based storage. SQLite is a popular local database option in Android due to its simplicity, efficiency, and integration with the Android framework.

SQLite Overview

SQLite is a lightweight, embedded database engine that is integrated directly into Android. Unlike server-based databases, SQLite is file-based and does not require a separate server process. It is ideal for apps that need a reliable, small-scale database solution without the overhead of a full database server.

Key Components

1. DatabaseHelper Class:

- **Purpose:** Manages database creation and version management.
- **Extends:** `SQLiteOpenHelper` class.
- **Methods:**
 - `onCreate()`: Called when the database is first created. Define table schemas here.
 - `onUpgrade()`: Called when the database version is incremented. Used to manage schema changes.

2. Database Operations:

- **Insert:** Adding new records to the database.
- **Read:** Retrieving data from the database.
- **Update:** Modifying existing records.
- **Delete:** Removing records from the database.

Implementation

1. DatabaseHelper Class

This class encapsulates all database-related operations.

```
package com.example.myapplication;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
```

```

import android.database.sqlite.SQLiteOpenHelper;

public class DatabaseHelper extends SQLiteOpenHelper {

    private static final String DATABASE_NAME = "my_database.db";
    private static final int DATABASE_VERSION = 1;

    private static final String TABLE_NAME = "my_table";
    private static final String COLUMN_ID = "id";
    private static final String COLUMN_NAME = "name";
    private static final String COLUMN_AGE = "age";

    private static final String TABLE_CREATE =
        "CREATE TABLE " + TABLE_NAME + " (" +
        COLUMN_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, "
+
        COLUMN_NAME + " TEXT, " +
        COLUMN_AGE + " INTEGER);";

    public DatabaseHelper(Context context) {
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL(TABLE_CREATE);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int
newVersion) {
        db.execSQL("DROP TABLE IF EXISTS " + TABLE_NAME);
        onCreate(db);
    }

    // Insert data
    public void insertData(String name, int age) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues values = new ContentValues();
        values.put(COLUMN_NAME, name);
        values.put(COLUMN_AGE, age);
        db.insert(TABLE_NAME, null, values);
        db.close();
    }

    // Read data
    public Cursor getData() {
        SQLiteDatabase db = this.getReadableDatabase();
        return db.rawQuery("SELECT * FROM " + TABLE_NAME, null);
    }

    // Update data
    public void updateData(int id, String name, int age) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues values = new ContentValues();
        values.put(COLUMN_NAME, name);
        values.put(COLUMN_AGE, age);
    }
}

```

```

        db.update(TABLE_NAME, values, COLUMN_ID + " = ?", new
String[] {String.valueOf(id)});
        db.close();
    }

    // Delete data
    public void deleteData(int id) {
        SQLiteDatabase db = this.getWritableDatabase();
        db.delete(TABLE_NAME, COLUMN_ID + " = ?", new
String[] {String.valueOf(id)});
        db.close();
    }
}

```

2. Activity Layout (XML)

Defines the user interface for interacting with the database:

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".SqliteActivity">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        android:layout_marginTop="80dp"
        android:padding="16dp">

        <EditText
            android:id="@+id/nameEditText"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Name" />

        <EditText
            android:id="@+id/ageEditText"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Age"
            android:inputType="number" />

        <Button
            android:id="@+id/insertButton"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Insert" />
    </LinearLayout>
</androidx.constraintlayout.widget.ConstraintLayout>

```

```

        <Button
            android:id="@+id/readButton"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Read" />

        <Button
            android:id="@+id/updateButton"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Update" />

        <Button
            android:id="@+id/deleteButton"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Delete" />

        <TextView
            android:id="@+id/dataTextView"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Data will appear here" />
    </LinearLayout>

</androidx.constraintlayout.widget.ConstraintLayout>

```

3. SQLiteActivity Class

Handles user interactions and invokes database operations:

```

package com.example.myapplication;

import android.database.Cursor;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class SQLiteActivity extends AppCompatActivity {
    private DatabaseHelper dbHelper;
    private EditText nameEditText;
    private EditText ageEditText;
    private TextView dataTextView;
    private Button insertButton;
    private Button readButton;
    private Button updateButton;
    private Button deleteButton;
}

```

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_sqlite);

    dbHelper = new DatabaseHelper(this);

    nameEditText = findViewById(R.id.nameEditText);
    ageEditText = findViewById(R.id.ageEditText);
    dataTextView = findViewById(R.id.dataTextView);
    insertButton = findViewById(R.id.insertButton);
    readButton = findViewById(R.id.readButton);
    updateButton = findViewById(R.id.updateButton);
    deleteButton = findViewById(R.id.deleteButton);

    insertButton.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            String name = nameEditText.getText().toString();
            int age =
Integer.parseInt(ageEditText.getText().toString());
            dbHelper.insertData(name, age);
        }
    });

    readButton.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            Cursor cursor = dbHelper.getData();
            StringBuilder data = new StringBuilder();
            if (cursor.moveToFirst()) {
                do {
                    int id =
cursor.getInt(cursor.getColumnIndex("id"));
                    String name =
cursor.getString(cursor.getColumnIndex("name"));
                    int age =
cursor.getInt(cursor.getColumnIndex("age"));
                    data.append("ID: ").append(id)
                        .append(", Name: ").append(name)
                        .append(", Age: ").append(age)
                        .append("\n");
                } while (cursor.moveToNext());
            }
            cursor.close();
            dataTextView.setText(data.toString());
        }
    });

    updateButton.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            // Assuming ID is 1 for update
            int id = 1;
            String name = nameEditText.getText().toString();

```

```

        int age =
Integer.parseInt(ageEditText.getText().toString());
        dbHelper.updateData(id, name, age);
    }
});

deleteButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        // Assuming ID is 1 for delete
        int id = 1;
        dbHelper.deleteData(id);
    }
});
}
}

```

Introduction to API, API Types, Introduction to JSON, Retrieving contents from remote server

What is an API?

API (Application Programming Interface) is a set of rules and tools that allow different software applications to communicate with each other. APIs define the methods and data formats that applications use to request and exchange information. APIs are crucial for enabling integration between different systems, services, and platforms.

API Types

1. REST (Representational State Transfer) APIs:

- **Description:** REST APIs are a set of rules for designing networked applications. They use HTTP requests to access and manipulate resources.
- **Characteristics:** Stateless, client-server architecture, uses standard HTTP methods (GET, POST, PUT, DELETE).
- **Data Format:** Typically JSON or XML.

2. SOAP (Simple Object Access Protocol) APIs:

- **Description:** SOAP is a protocol for exchanging structured information in the implementation of web services. It relies on XML-based messaging.
- **Characteristics:** Includes built-in error handling, security features, and a formal contract for service definition.
- **Data Format:** XML.

3. GraphQL APIs:

- **Description:** A query language for APIs that allows clients to request specific data. Unlike REST, which provides fixed endpoints, GraphQL allows clients to specify exactly what data they need.
- **Characteristics:** Flexible queries, single endpoint for all operations.

- **Data Format:** JSON.
- 4. **gRPC (gRPC Remote Procedure Calls) APIs:**
 - **Description:** A high-performance, open-source RPC framework that uses HTTP/2 for transport and Protocol Buffers as the serialization mechanism.
 - **Characteristics:** Supports multiple languages, efficient and fast communication.
 - **Data Format:** Protocol Buffers (binary format).

Introduction to JSON

What is JSON?

JSON (JavaScript Object Notation) is a lightweight data interchange format that's easy for humans to read and write and easy for machines to parse and generate. It is often used to transmit data between a server and a web application as text.

JSON Structure

- **Objects:** Key-value pairs enclosed in curly braces {}.
- **Arrays:** Ordered lists of values enclosed in square brackets [].
- **Values:** Can be strings, numbers, objects, arrays, `true`, `false`, or `null`.

Example of JSON:

```
{
  "name": "John Doe",
  "age": 30,
  "isStudent": false,
  "courses": ["Math", "Science"],
  "address": {
    "street": "123 Main St",
    "city": "Anytown"
  }
}
```

Retrieving Contents from a Remote Server

Volley is an Android networking library developed by Google. It provides an efficient and flexible way to handle network requests and responses, simplifying common tasks such as parsing JSON data, handling image loading, and managing network queues.

Features of Volley

- **Easy to Use:** Simplifies making network requests with built-in methods for GET, POST, PUT, DELETE, and other HTTP operations.
- **Automatic Request Queuing:** Manages a queue of network requests and prioritizes them.

- **Image Loading:** Includes built-in support for loading images from the network into `ImageView`.
- **Caching:** Provides request and response caching to reduce network traffic and improve performance.
- **Error Handling:** Includes mechanisms to handle network errors and response errors.

Implementing Volley in Android

1. Add Volley Dependency

Include Volley in your `build.gradle` file:

```
dependencies {  
    implementation 'com.android.volley:volley:1.2.1'  
}
```

2. Initialize RequestQueue

You need a `RequestQueue` to manage and execute network requests. Typically, you initialize it in your `Application` class.

```
import com.android.volley.RequestQueue;  
import com.android.volley.toolbox.Volley;  
  
public class MyApp extends Application {  
    private static RequestQueue requestQueue;  
  
    @Override  
    public void onCreate() {  
        super.onCreate();  
        requestQueue = Volley.newRequestQueue(this);  
    }  
  
    public static RequestQueue getRequestQueue() {  
        return requestQueue;  
    }  
}
```

3. Make a Network Request

GET Request Example:

```
import android.os.Bundle;  
import android.util.Log;  
import android.widget.TextView;  
  
import androidx.appcompat.app.AppCompatActivity;  
  
import com.android.volley.Request;
```



```

import com.android.volley.RequestQueue;
import com.android.volley.Response;
import com.android.volley.VolleyError;
import com.android.volley.toolbox.JsonArrayRequest;
import com.android.volley.toolbox.Volley;

import org.json.JSONArray;
import org.json.JSONException;
import org.json.JSONObject;

public class MainActivity extends AppCompatActivity {

    private static final String URL =
"https://jsonplaceholder.typicode.com/users";
    private TextView textView;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        textView = findViewById(R.id.textview);

        RequestQueue requestQueue = MyApp.getRequestQueue();

        JsonArrayRequest jsonArrayRequest = new JsonArrayRequest(
            Request.Method.GET,
            URL,
            null,
            new Response.Listener<JSONArray>() {
                @Override
                public void onResponse(JSONArray response) {
                    StringBuilder sb = new StringBuilder();
                    for (int i = 0; i < response.length(); i++) {
                        try {
                            JSONObject user =
response.getJSONObject(i);

                            int id = user.getInt("id");
                            String name = user.getString("name");
                            String email = user.getString("email");

                            sb.append("ID: ").append(id)
                                .append(", Name: ").append(name)
                                .append(", Email: ").append(email)
                                .append("\n");
                        } catch (JSONException e) {
                            e.printStackTrace();
                        }
                    }
                    textView.setText(sb.toString());
                }
            },
            new Response.ErrorListener() {
                @Override
                public void onErrorResponse(VolleyError error) {
                    Log.e("VOLLEY_ERROR", error.toString());
                }
            }
        );
    }
}

```

```

        }
    };

    requestQueue.add(jsonArrayRequest);
}
}

```

Sending contents to remote server

To send contents to a remote server from an Android application, you can use several methods, including **Volley**, **Retrofit**, and **OkHttp**.

1. Using Volley

Volley is useful for sending both simple and complex data to a server. Below is an example of how to send a `POST` request with JSON data using Volley.

1. Add Dependencies:

In your `build.gradle` file, include the Volley dependency:

```

dependencies {
    implementation 'com.android.volley:volley:1.2.1'
}

```

3. Create a POST Request:

Send JSON data to a server using Volley:

```

import android.os.Bundle;
import android.util.Log;
import android.widget.Button;
import android.widget.EditText;
import androidx.appcompat.app.AppCompatActivity;
import com.android.volley.Request;
import com.android.volley.RequestQueue;
import com.android.volley.Response;
import com.android.volley.VolleyError;
import com.android.volley.toolbox.JsonObjectRequest;
import com.android.volley.toolbox.Volley;
import org.json.JSONObject;

public class MainActivity extends AppCompatActivity {

    private static final String URL =
    "https://jsonplaceholder.typicode.com/posts";
    private EditText titleEditText;

```

```

private EditText bodyEditText;
private Button sendButton;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    titleEditText = findViewById(R.id.titleEditText);
    bodyEditText = findViewById(R.id.bodyEditText);
    sendButton = findViewById(R.id.sendButton);

    RequestQueue requestQueue = Volley.newRequestQueue(this);

    sendButton.setOnClickListener(v -> {
        String title = titleEditText.getText().toString();
        String body = bodyEditText.getText().toString();

        JSONObject jsonBody = new JSONObject();
        try {
            jsonBody.put("title", title);
            jsonBody.put("body", body);
            jsonBody.put("userId", 1); // Example userId
        } catch (Exception e) {
            e.printStackTrace();
        }

        JsonObjectRequest jsonObjectRequest = new
JsonObjectRequest(
            Request.Method.POST,
            URL,
            jsonBody,
            response -> Log.d("VolleyResponse",
response.toString()),
            error -> Log.e("VolleyError", error.toString())
        );

        requestQueue.add(jsonObjectRequest);
    });
}
}

```

2. Using Retrofit

Retrofit simplifies making network requests and handling responses. Here's how to send JSON data using Retrofit.

1. Add Dependencies:

Include Retrofit in your build.gradle:

```

dependencies {
    implementation 'com.squareup.retrofit2:retrofit:2.9.0'
}

```

```
        implementation 'com.squareup.retrofit2:converter-gson:2.9.0'  
    }
```

2. Define the API Interface:

Create an interface for your API endpoints:

```
import retrofit2.Call;  
import retrofit2.http.Body;  
import retrofit2.http.POST;  
  
public interface ApiService {  
    @POST("posts")  
    Call<Post> createPost(@Body Post post);  
}
```

3. Create the Retrofit Instance and Send Data:

```
import android.os.Bundle;
import android.util.Log;
import android.widget.Button;
import android.widget.EditText;
import androidx.appcompat.app.AppCompatActivity;
import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;
import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;

public class MainActivity extends AppCompatActivity {

    private static final String BASE_URL =
"https://jsonplaceholder.typicode.com/";
    private EditText titleEditText;
    private EditText bodyEditText;
    private Button sendButton;
    private ApiService apiService;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        titleEditText = findViewById(R.id.titleEditText);
        bodyEditText = findViewById(R.id.bodyEditText);
        sendButton = findViewById(R.id.sendButton);

        Retrofit retrofit = new Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build();

        apiService = retrofit.create(ApiService.class);

        sendButton.setOnClickListener(v -> {
            String title = titleEditText.getText().toString();
            String body = bodyEditText.getText().toString();

            Post post = new Post(title, body, 1); // Example userId
            Call<Post> call = apiService.createPost(post);
            call.enqueue(new Callback<Post>() {
                @Override
                public void onResponse(Call<Post> call, Response<Post>
response) {
                    if (response.isSuccessful()) {
                        Log.d("RetrofitResponse",
response.body().toString());
                    } else {
                        Log.e("RetrofitError", "Error: " +
response.message());
                    }
                }
            })
        })
    }
}
```

```

        @Override
        public void onFailure(Call<Post> call, Throwable t) {
            Log.e("RetrofitError", t.toString());
        }
    });
}
}
}

```

4. Define the Data Model:

```

public class Post {
    private String title;
    private String body;
    private int userId;

    public Post(String title, String body, int userId) {
        this.title = title;
        this.body = body;
        this.userId = userId;
    }

    // Getters and setters
}

```

3. Using OkHttp

1. Add Dependencies:

In your `build.gradle` file:

```

dependencies {
    implementation 'com.squareup.okhttp3:okhttp:5.0.0-alpha.2'
}

```

1. Create and Send a POST Request:

```

import android.os.Bundle;
import android.util.Log;
import android.widget.Button;
import android.widget.EditText;
import androidx.appcompat.app.AppCompatActivity;
import okhttp3.MediaType;
import okhttp3.OkHttpClient;
import okhttp3.Request;
import okhttp3.RequestBody;
import okhttp3.Call;
import okhttp3.Callback;

```

```

import okhttp3.Response;

import java.io.IOException;

public class MainActivity extends AppCompatActivity {

    private static final String URL =
"https://jsonplaceholder.typicode.com/posts";
    private EditText titleEditText;
    private EditText bodyEditText;
    private Button sendButton;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        titleEditText = findViewById(R.id.titleEditText);
        bodyEditText = findViewById(R.id.bodyEditText);
        sendButton = findViewById(R.id.sendButton);

        OkHttpClient client = new OkHttpClient();

        sendButton.setOnClickListener(v -> {
            String title = titleEditText.getText().toString();
            String body = bodyEditText.getText().toString();

            MediaType JSON = MediaType.get("application/json;
charset=utf-8");
            String jsonBody = "{ \"title\": \"" + title + "\",
\"body\": \"" + body + "\", \"userId\": 1 }";
            RequestBody body = RequestBody.create(jsonBody, JSON);

            Request request = new Request.Builder()
                .url(URL)
                .post(body)
                .build();

            client.newCall(request).enqueue(new Callback() {
                @Override
                public void onFailure(Call call, IOException e) {
                    Log.e("OkHttpError", e.toString());
                }

                @Override
                public void onResponse(Call call, Response response)
throws IOException {
                    if (response.isSuccessful()) {
                        Log.d("OkHttpResponse",
response.body().string());
                    } else {
                        Log.e("OkHttpError", response.message());
                    }
                }
            });
        });
    }
}

```

```
}
```

Implementing Google Maps in android application

Implementing Google Maps in an Android application involves several steps. You'll need to set up your project to use the Google Maps SDK, get an API key from the Google Cloud Console, and then integrate the map into your app. Here's a step-by-step guide to help you through the process:

1. Set Up Your Project

1.1 Add Google Maps SDK Dependency

In your `build.gradle` (app-level) file, add the following dependency:

```
dependencies {  
    implementation 'com.google.android.gms:play-services-maps:19.1.0'  
}
```

1.2 Add Permissions and Metadata in `AndroidManifest.xml`

Add the following permissions and metadata to your `AndroidManifest.xml`:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:tools="http://schemas.android.com/tools">  
  
    <application  
        ...>  
  
        <!-- Metadata for Google Maps API Key -->  
        <meta-data  
            android:name="com.google.android.geo.API_KEY"  
            android:value="YOUR_API_KEY"/>  
  
        <!-- Permission for accessing the Internet -->  
        <uses-permission android:name="android.permission.INTERNET"/>  
  
        <!-- Permission for accessing device location -->  
        <uses-permission  
            android:name="android.permission.ACCESS_FINE_LOCATION"/>  
  
        <!-- Permission for accessing network state -->  
        <uses-permission  
            android:name="android.permission.ACCESS_NETWORK_STATE"/>  
  
        <activity  
            android:name=".MapsActivity"  
            android:label="@string/app_name"  
            android:theme="@style/Theme.MyApplication">
```



```
<intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
</activity>

</application>

</manifest>
```

Replace `YOUR_API_KEY` with the API key you obtained from the Google Cloud Console.

2. Get Your Google Maps API Key

2.1 Obtain API Key

1. Go to the Google Cloud Console (<https://console.cloud.google.com/>) or Developers Console (<https://console.developers.google.com>).
2. Create a new project or select an existing one.
3. Navigate to **APIs & Services > Credentials**.
4. Click **Create credentials** and select **API key**.
5. Restrict your API key as needed for security purposes.

3. Create the Maps Activity

3.1 Add a Map Fragment to Your Layout

Create a new layout file `activity_maps.xml` with a `MapFragment` or `MapView`:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MapsActivity">

    <fragment
        android:id="@+id/map"
        android:name="com.google.android.gms.maps.SupportMapFragment"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

</RelativeLayout>
```

3.2 Create MapsActivity.java

Create a new Java file `MapsActivity.java`:

```

import android.os.Bundle;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import com.google.android.gms.maps.CameraUpdateFactory;
import com.google.android.gms.maps.GoogleMap;
import com.google.android.gms.maps.OnMapReadyCallback;
import com.google.android.gms.maps.SupportMapFragment;
import com.google.android.gms.maps.model.LatLng;
import com.google.android.gms.maps.model.MarkerOptions;
import com.google.android.gms.maps.model.Marker;

public class MapsActivity extends AppCompatActivity implements
OnMapReadyCallback {

    private GoogleMap mMap;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_maps);

        // Obtain the SupportMapFragment and get notified when the map is
        // ready to be used.
        SupportMapFragment mapFragment = (SupportMapFragment)
getSupportFragmentManager()
        .findFragmentById(R.id.map);
        mapFragment.getMapAsync(this);
    }

    @Override
    public void onMapReady(@NonNull GoogleMap googleMap) {
        mMap = googleMap;

        // Add a marker at Sydney and move the camera
        LatLng sydney = new LatLng(-34, 151);
        mMap.addMarker(new MarkerOptions().position(sydney).title("Marker in Sydney"));
        mMap.moveCamera(CameraUpdateFactory.newLatLng(sydney));
    }
}

```

Additional Features

- **Adding Markers:**

To add more markers to the map:

```
LatLng location = new LatLng(latitude, longitude);  
mMap.addMarker(new MarkerOptions().position(location).title("Marker Title"));
```

- **Moving the Camera:**

To move the camera to a specific location:

```
LatLng newLocation = new LatLng(latitude, longitude);  
mMap.moveCamera(CameraUpdateFactory.newLatLngZoom(newLocation, zoomLevel));
```

- **Handling Map Events:**

You can add listeners for various map events, such as user interactions:

```
mMap.setOnMapClickListener(latLng -> {  
    // Handle map click  
    mMap.addMarker(new MarkerOptions().position(latLng).title("New Marker"));  
});
```

Procedure for publishing application on Google Play Store.

Sign Your APK or AAB: Use Android Studio or the command line to sign your APK or AAB.

Using Android Studio:

1. Go to Build > Generate Signed Bundle / APK.
2. Select either APK or Android App Bundle (AAB) and click Next.
3. Enter your keystore and key information, then click Finish.

Prepare Your Play Store Listing

a. Create a Google Play Developer Account

- **Sign Up:** Go to the Google Play Console and sign up for a developer account.
- **Pay Registration Fee:** There is a one-time registration fee of \$25.

b. Prepare Listing Details

- **App Title and Description:** Write a clear and compelling app title and description.
- **Screenshots and Graphics:** Prepare high-quality screenshots, app icons, feature graphics, and promotional videos.
- **Category:** Choose the appropriate category for your app.
- **Privacy Policy:** Provide a link to your app's privacy policy if it collects user data.

3. Upload Your App to the Play Store

a. Create a New Application

- **Go to Play Console:** Log in to the Google Play Console.
- **Create Application:** Click on `Create Application`, select your default language, and enter the title of your app.

b. Fill Out the Store Listing

- **Complete All Required Fields:** Provide the app title, description, screenshots, and other required details.
- **Content Rating:** Complete the content rating questionnaire to help Google determine the appropriate age group for your app.
- **Pricing and Distribution:** Set the pricing and distribution countries for your app.

c. Upload Your APK or AAB

- **Navigate to the 'Release' Section:** Go to the `Release` section in the Play Console.
- **Create a Release:** Click on `Create New Release`, then upload your APK or AAB.
- **Review and Rollout:** Review your release information and then click `Save and Review Release`. Finally, click `Start Rollout to Production` to publish your app.

4. Submit for Review

- **Review Process:** Google will review your app for compliance with their policies. This process can take a few hours to a few days.
- **Respond to Feedback:** Be prepared to address any feedback or issues raised by Google during the review.

Signed Vs. Unsigned APK

A comparison between a signed APK and an unsigned APK:

Aspect	Signed APK	Unsigned APK
Definition	An APK that has been digitally signed with a cryptographic key, typically from a keystore.	An APK that has not been signed with any cryptographic key.
Security	Provides security through digital signatures to ensure the integrity and authenticity of the APK.	Lacks a digital signature, making it less secure and vulnerable to tampering.

Aspect	Signed APK	Unsigned APK
Installation	Can be installed on devices and distributed via the Google Play Store.	Cannot be installed on devices or uploaded to the Google Play Store without signing.
Distribution	Required for app distribution on the Google Play Store.	Not suitable for distribution on the Google Play Store or for use on most devices.
Verification	Verifies the authenticity of the app and ensures it has not been altered since it was signed.	No verification of authenticity or integrity.
Permissions	Signed APKs can have permissions granted based on the signature's trust level.	Unsigned APKs lack the capability to request or enforce permissions properly.
Update Mechanism	Allows for version management and updates through the Play Store, preserving user data and settings.	Updates cannot be managed through the Play Store, and existing installations cannot be updated.
Release Process	Essential for releasing apps to users and publishing on the Play Store.	Primarily used for development and testing purposes, not suitable for release.
Key Management	Requires managing a keystore and key for signing, which provides a way to secure and identify the developer.	No key management needed as no signing occurs.
Legal and Policy Compliance	Complies with Google Play Store policies and other app distribution regulations.	Non-compliant with most app distribution policies and regulations.